

Survival Analysis Using Titanic Dataset

In this project, we explore a cleaned version of the Titanic dataset to predict whether a passenger survived or not using logistic regression.

This dataset is often used as an entry point for understanding classification in machine learning.

We'll go through a full exploratory analysis, clean the data, visualize trends, and build a predictive model.

Step 1: Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set_style('whitegrid')
%matplotlib inline
```

Loading the Dataset

Let's import the training dataset into a DataFrame and take a look at the first few rows.

```
train = pd.read_csv("C:/Users/rajpu/Downloads/titanic_train.csv")
```

```
train.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cummings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
Futrelle, Mrs. Jacques Heath (Lily)												

Exploratory Data Analysis

Let's begin some exploratory data analysis! We'll start by checking out missing data!

Missing Data

We can use seaborn to create a simple heatmap to see where we are missing data!

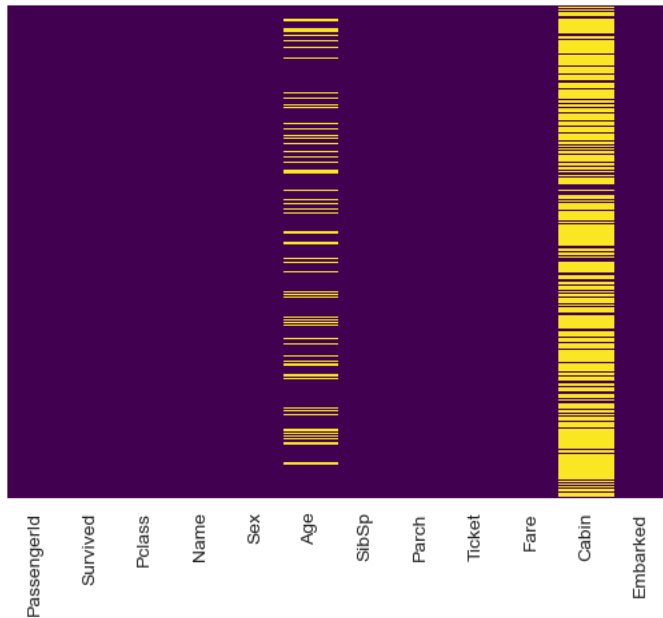
```
train.isnull()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	False	False	False	False	False	False	False	False	False	False	True	False
1	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	True	False
3	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	True	False
...	...	...	...	...	...	...	...	...	...	...	...	...
886	False	False	False	False	False	False	False	False	False	False	True	False
887	False	False	False	False	False	False	False	False	False	False	False	False
888	False	False	False	False	False	True	False	False	False	False	True	False
889	False	False	False	False	False	False	False	False	False	False	False	False
890	False	False	False	False	False	False	False	False	False	False	True	False

891 rows × 12 columns

```
sns.heatmap(train.isnull(),yticklabels=False,cbar=False,cmap='viridis')
```

<Axes: >

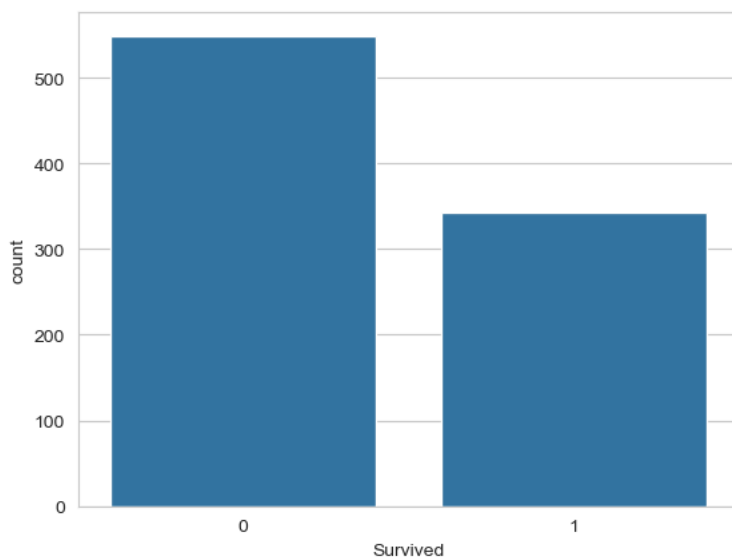


Roughly 20 percent of the Age data is missing. The proportion of Age missing is likely small enough for reasonable replacement with some form of imputation. Looking at the Cabin column, it looks like we are just missing too much of that data to do something useful with at a basic level. We'll probably drop this later, or change it to another feature like "Cabin Known: 1 or 0"

Let's continue on by visualizing some more of the data! Check out the video for full explanations over these plots, this code is just to serve as reference.

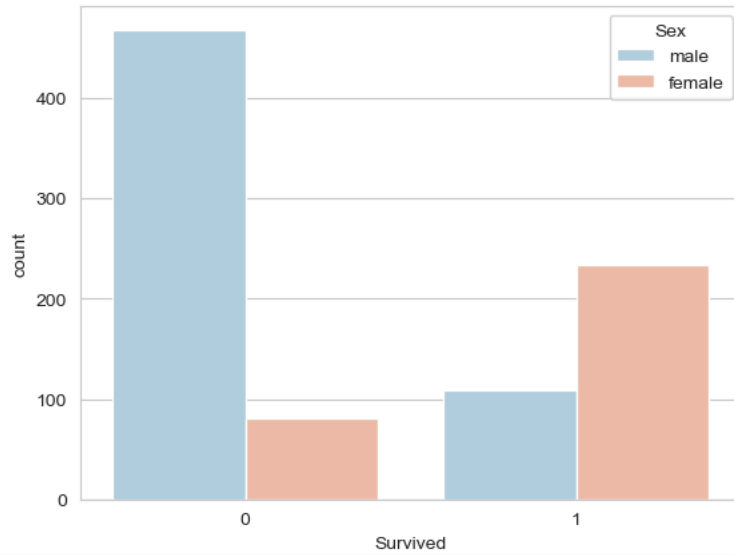
```
sns.set_style('whitegrid')
sns.countplot(x='Survived',data=train)
```

<Axes: xlabel='Survived', ylabel='count'>



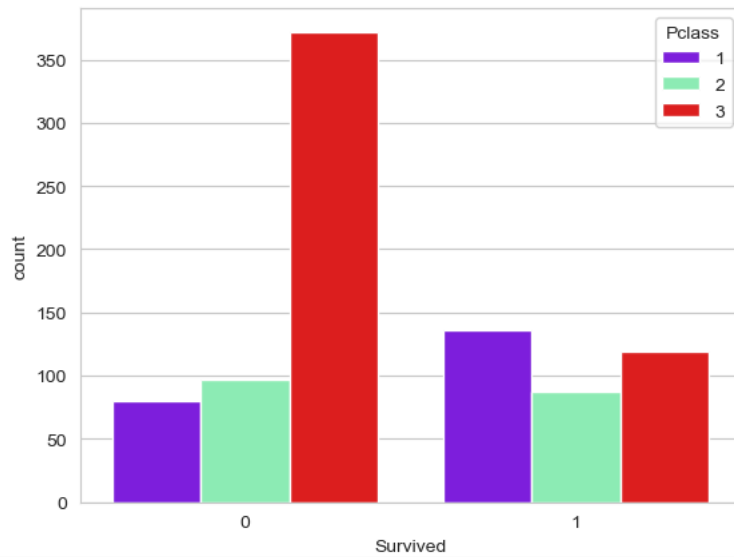
```
sns.set_style('whitegrid')
sns.countplot(x='Survived',hue='Sex',data=train,palette='RdBu_r')
```

<Axes: xlabel='Survived', ylabel='count'>



```
sns.set_style('whitegrid')  
sns.countplot(x='Survived', hue='Pclass', data=train, palette='rainbow')
```

<Axes: xlabel='Survived', ylabel='count'>



```
sns.distplot(train['Age'].dropna(), kde=False, color='darkred', bins=40)
```

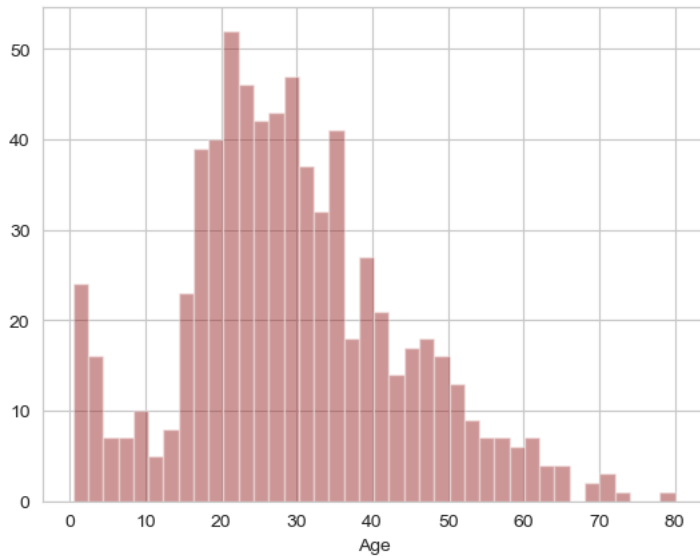
↗ C:\Users\rajpu\AppData\Local\Temp\ipykernel_10080\2002818437.py:1: UserWarning:

``distplot` is a deprecated function and will be removed in seaborn v0.14.0.`

Please adapt your code to use either ``displot`` (a figure-level function with similar flexibility) or ``histplot`` (an axes-level function for histograms).

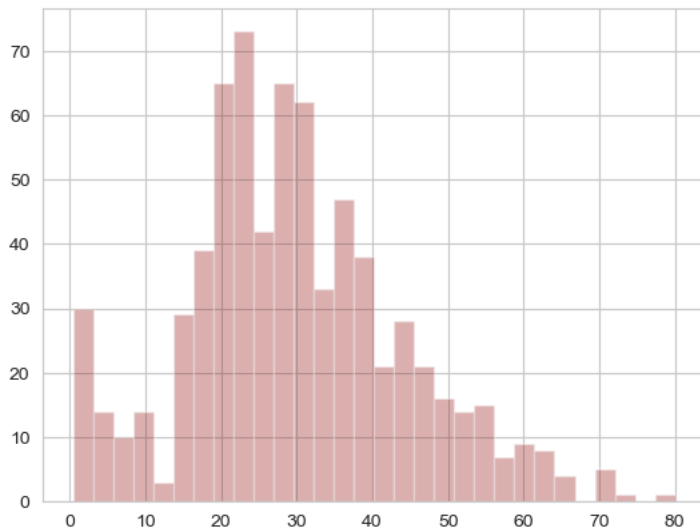
For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(train['Age'].dropna(),kde=False,color='darkred',bins=40)
<Axes: xlabel='Age'>
```



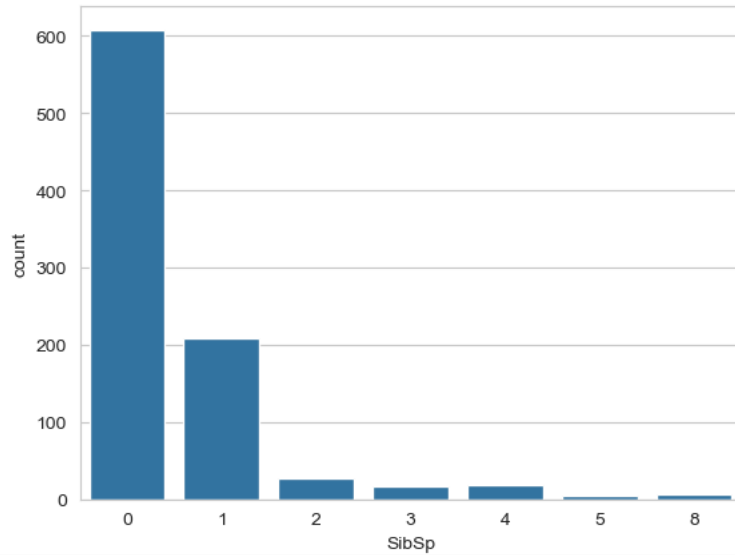
```
train['Age'].hist(bins=30,color='darkred',alpha=0.3)
```

↗ <Axes: >



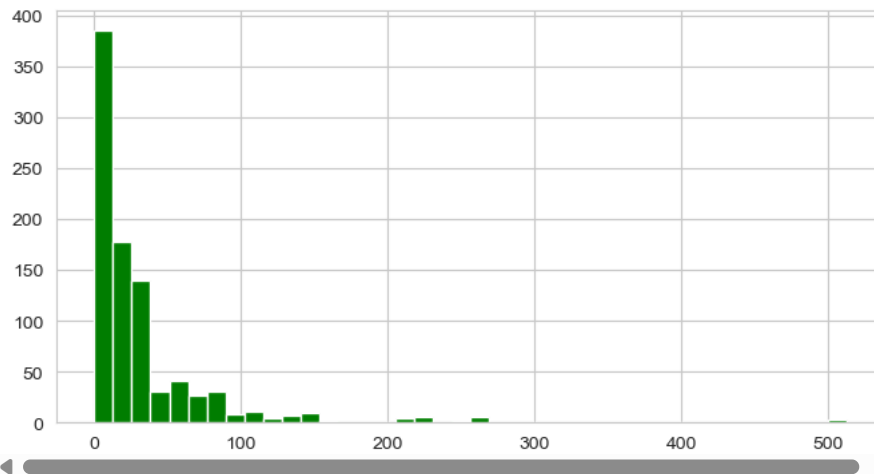
```
sns.countplot(x='SibSp',data=train)
```

 <Axes: xlabel='SibSp', ylabel='count'>



```
train['Fare'].hist(color='green',bins=40,figsize=(8,4))
```

 <Axes: >



✓ Data Cleaning

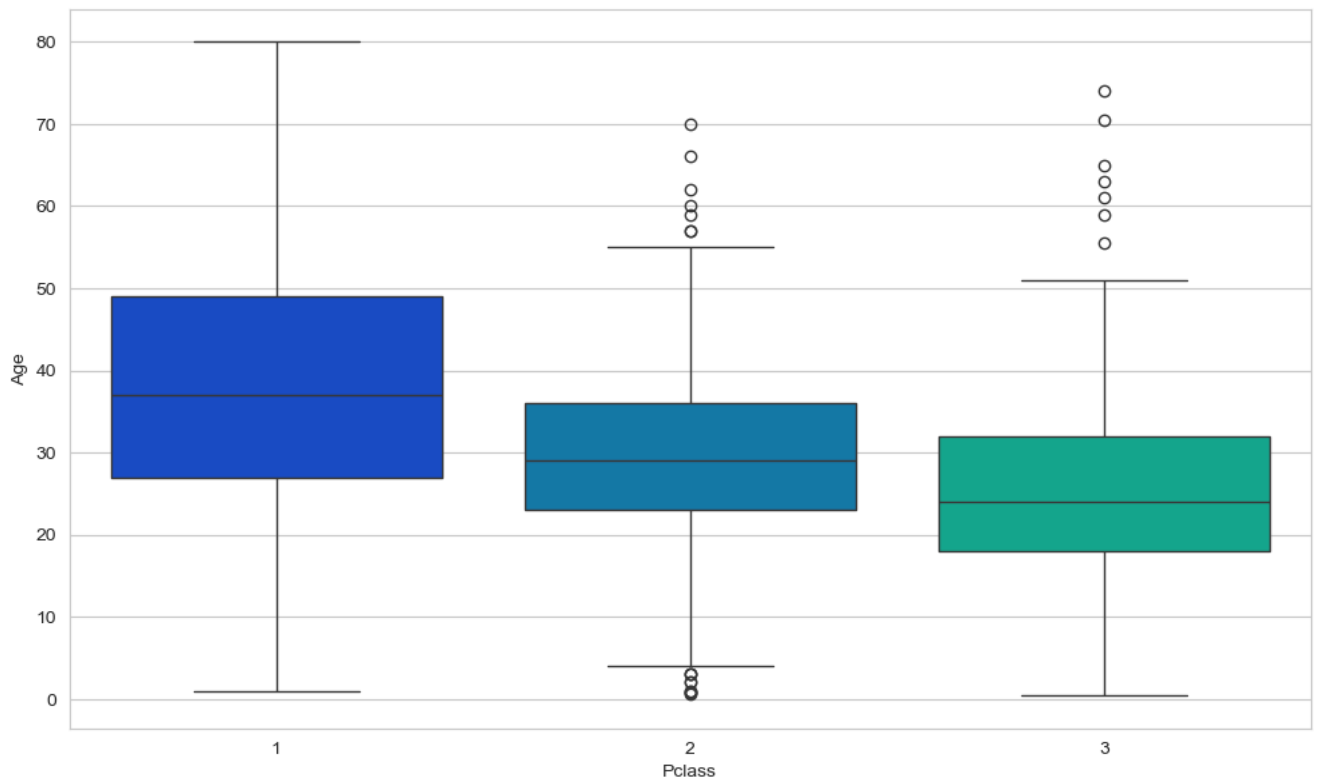
We want to fill in missing age data instead of just dropping the missing age data rows. One way to do this is by filling in the mean age of all the passengers (imputation). However we can be smarter about this and check the average age by passenger class. For example:

```
plt.figure(figsize=(12, 7))
sns.boxplot(x='Pclass',y='Age',data=train,palette='winter')
```

C:\Users\rajpu\AppData\Local\Temp\ipykernel_10080\1683851715.py:2: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set

```
sns.boxplot(x='Pclass', y='Age', data=train, palette='winter')
<Axes: xlabel='Pclass', ylabel='Age'>
```



We can see the wealthier passengers in the higher classes tend to be older, which makes sense. We'll use these average age values to impute based on Pclass for Age.

```
def impute_age(cols):
    Age = cols[0]
    Pclass = cols[1]

    if pd.isnull(Age):

        if Pclass == 1:
            return 37

        elif Pclass == 2:
            return 29

        else:
            return 24

    else:
        return Age
```

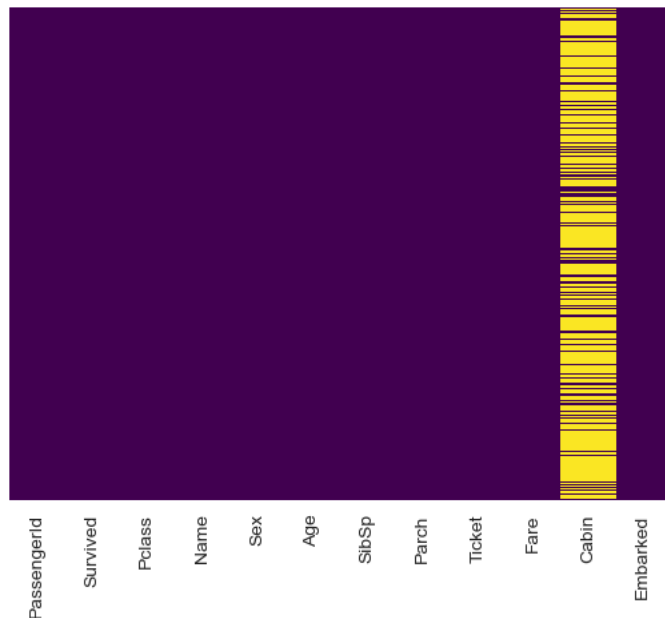
Now apply that function!

```
train['Age'] = train[['Age', 'Pclass']].apply(impute_age,axis=1)
```

C:\Users\rajpu\AppData\Local\Temp\ipykernel_10080\822839471.py:2: FutureWarning: Series.__getitem__ treating keys as positions
Age = cols[0]
C:\Users\rajpu\AppData\Local\Temp\ipykernel_10080\822839471.py:3: FutureWarning: Series.__getitem__ treating keys as positions
Pclass = cols[1]

Now let's check that heat map again!

```
sns.heatmap(train.isnull(),yticklabels=False,cbar=False,cmap='viridis')
```

 <Axes: >


Great! Let's go ahead and drop the Cabin column and the row in Embarked that is NaN.

```
train.drop('Cabin',axis=1,inplace=True)
```

```
train.head()
```



	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	S
Futrelle, Mrs. Jacques Heath (Lily May											

```
train.dropna(inplace=True)
```

✓ Converting Categorical Features

We'll need to convert categorical features to dummy variables using pandas! Otherwise our machine learning algorithm won't be able to directly take in those features as inputs.

```
train.info()
```



```
<class 'pandas.core.frame.DataFrame'>
Index: 889 entries, 0 to 890
Data columns (total 11 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   PassengerId  889 non-null    int64
1   Survived     889 non-null    int64
2   Pclass       889 non-null    int64
3   Name         889 non-null    object
4   Sex          889 non-null    object
5   Age          889 non-null    float64
6   SibSp        889 non-null    int64
7   Parch        889 non-null    int64
8   Ticket       889 non-null    object
9   Fare         889 non-null    float64
10  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(4)
memory usage: 83.3+ KB
```

```
pd.get_dummies(train['Embarked'],drop_first=True).head()
```



```

      Q      S
0  False  True
1  False  False
2  False  True
3  False  True
4  False  True

```

```
sex = pd.get_dummies(train['Sex'],drop_first=True)
embark = pd.get_dummies(train['Embarked'],drop_first=True)
```

```
train.drop(['Sex', 'Embarked', 'Name', 'Ticket'],axis=1,inplace=True)
```

```
train.head()
```



```

  PassengerId  Survived  Pclass   Age  SibSp  Parch    Fare
0            1         0       3  22.0     1     0   7.2500
1            2         1       1  38.0     1     0  71.2833
2            3         1       3  26.0     0     0   7.9250
3            4         1       1  35.0     1     0  53.1000
4            5         0       3  35.0     0     0   8.0500

```

```
train = pd.concat([train,sex,embark],axis=1)
```

```
train.head()
```



```

  PassengerId  Survived  Pclass   Age  SibSp  Parch    Fare  male     Q     S
0            1         0       3  22.0     1     0   7.2500   True  False  True
1            2         1       1  38.0     1     0  71.2833  False  False  False
2            3         1       3  26.0     0     0   7.9250  False  False  True
3            4         1       1  35.0     1     0  53.1000  False  False  True
4            5         0       3  35.0     0     0   8.0500   True  False  True

```

Great! Our data is ready for our model!

✓ Building a Logistic Regression model

Let's start by splitting our data into a training set and test set (there is another test.csv file that you can play around with in case you want to use all this data for training).

Train Test Split

```
train.drop('Survived',axis=1).head()
```



```

  PassengerId  Pclass   Age  SibSp  Parch    Fare  male     Q     S
0            1       3  22.0     1     0   7.2500   True  False  True
1            2       1  38.0     1     0  71.2833  False  False  False
2            3       3  26.0     0     0   7.9250  False  False  True
3            4       1  35.0     1     0  53.1000  False  False  True
4            5       3  35.0     0     0   8.0500   True  False  True

```

```
train['Survived'].head()
```



```

0    0
1    1
2    1
3    1
4    0
Name: Survived, dtype: int64

```



```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(train.drop('Survived',axis=1),
                                                    train['Survived'], test_size=0.30,
                                                    random_state=101)
```

✓ Training and Predicting

```
from sklearn.linear_model import LogisticRegression
```

```
logmodel = LogisticRegression()
logmodel.fit(X_train,y_train)
```

 C:\Users\rajpu\anaconda3\Lib\site-packages\sklearn\linear_model_logistic.py:469: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

▼ LogisticRegression ⓘ ?

```
LogisticRegression()
```

```
predictions = logmodel.predict(X_test)
```

```
from sklearn.metrics import confusion_matrix
```

```
accuracy=confusion_matrix(y_test,predictions)
```

```
accuracy
```

 array([[149, 14],
[39, 65]], dtype=int64)

```
from sklearn.metrics import accuracy_score
```

```
accuracy=accuracy_score(y_test,predictions)
accuracy
```

 0.8014981273408239

```
predictions
```